

GPTQ 量化算法

目 录

1 引言：RTN 的不足与 GPTQ 的动机	2
2 训练后量化 (PTQ)	2
3 层重构目标	2
3.1 动机	2
3.2 形式化	2
4 Hessian 矩阵	3
4.1 推导	3
4.2 回看	3
5 误差补偿	3
5.1 核心思想	3
5.2 公式	3
6 Cholesky 分解	4
6.1 为什么需要 Cholesky	4
6.2 优势	4
7 阻尼与数值稳定性	4
7.1 问题	4
8 分块计算	4
8.1 动机	4
8.2 策略	4
9 逐层量化流程	5
10 代码接口	5
11 实战：手动算 GPTQ	5
12 本章你将学会	7
13 小结	7

1 引言：RTN 的不足与 GPTQ 的动机

上一章我们学习了 RTN 量化：对每个权重组独立计算 scale，直接舍入。RTN 简单快速，但有一个根本缺陷：它把每个权重视为独立的，量化第 i 列的误差不会影响第 $i+1$ 列的决策。实际上，权重矩阵的各列通过输入激活产生关联，一列的误差会影响其他列的输出，进而影响整体精度。

GPTQ (Generalized Post-Training Quantization) 用二阶信息解决这个问题。核心思想是：量化第 i 列后，根据输入激活的统计特性，调整尚未量化的列，使后续列尽可能抵消当前列引入的输出误差。这种误差补偿让 INT4 下的精度损失大幅降低。

直觉 想象你在搭积木：RTN 是每块积木独立摆放，一块歪了不影响下一块；GPTQ 是每放一块就检查整体，如果这块歪了，调整下一块的位置来补偿。最终整体结构更稳固。

2 训练后量化 (PTQ)

PTQ (Post-Training Quantization, 训练后量化) 是指在模型训练完成后，用少量校准数据确定量化参数，不需要重新训练。与之相对的是 QAT (Quantization-Aware Training, 量化感知训练)，在训练过程中模拟量化误差，精度更好但成本高。

GPTQ 属于 PTQ：我们收集一批校准文本，前向传播时记录每一层线性层的输入激活，用这些激活构造 Hessian 矩阵来指导量化。本实验使用 256 条校准文本，最多采集 4096 个 token 的激活。

3 层重构目标

3.1 动机

量化的最终目标不是让权重本身尽量准确，而是让量化后的输出尽量接近原始输出。对于线性层 $Y = XW^T$ ，我们希望 XQ^T 尽量接近 XW^T ，其中 Q 是量化后的权重。

3.2 形式化

层重构目标为：

$$\min_Q \|XW^T - XQ^T\|_F^2 = \min_Q \|X(W - Q)^T\|_F^2 \quad (3.1)$$

其中 $X \in \mathbb{R}^{N \times d_{in}}$ 是输入激活， $W, Q \in \mathbb{R}^{d_{out} \times d_{in}}$ 是原始和量化后的权重， $\|\cdot\|_F$ 是 Frobenius 范数。

这个目标只涉及一层的前向输出误差，不考虑对后续层的影响。逐层独立量化是 GPTQ 的标准做法，简单有效。

关键观察： $X(W - Q)^T$ 中， X 的不同列对输出误差的贡献不同。如果某列的激活值很大（该通道很“活跃”），该列的量化误差对输出影响也大。这种“通道重要性”由 Hessian 矩阵的对角元素刻画。

4 Hessian 矩阵

4.1 推导

对层重构目标展开：

$$\|X(W - Q)^T\|_F^2 = \sum_j \|X(w_j - q_j)\|^2 = \sum_j (w_j - q_j)^T X^T X (w_j - q_j) \quad (4.1)$$

其中 w_j 和 q_j 是 W 和 Q 的第 j 列。定义 Hessian 矩阵：

$$H = \frac{2}{N} X^T X \quad (4.2)$$

则每列的误差贡献为 $(\frac{N}{2})(w_j - q_j)^T H (w_j - q_j)$ 。 H 的对角元素 $H_{j,j} = \frac{2}{N} \sum_n X_{n,j}^2$ 反映第 j 个通道的活跃程度，非对角元素 $H_{i,j} = \frac{2}{N} \sum_n X_{n,i} X_{n,j}$ 反映通道 i 和 j 之间的相关性。

直觉 Hessian 像一张“重要性地图”：对角线告诉你每个通道有多重要，非对角线告诉你通道之间的关系有多紧密。量化一个重要通道时误差大，Hessian 会指导 GPTQ 把误差往不相关的通道上“推”，减轻整体影响。

4.2 回看

Hessian 只依赖于输入激活 X ，与权重 W 无关。这意味着我们可以提前用校准数据收集激活，一次性算出 H ，然后对每一层独立执行 GPTQ 量化。

5 误差补偿

5.1 核心理想

GPTQ 逐列量化权重。量化第 i 列后，用 Hessian 逆矩阵的信息调整剩余列，使后续列的输出误差最小化。

5.2 公式

设 $H^{-1} = U^T U$ (U 为上三角矩阵，来自 Cholesky 分解)。量化第 i 列时：

$$e_i = \frac{w_i - q_i}{U_{i,i}} \quad (5.1)$$

$$W_{:,i} \leftarrow W_{:,i} - e_i U_{i,:} \quad (5.2)$$

其中 w_i 是第 i 列的原始权重， q_i 是量化后的权重， e_i 是补偿向量。第一个公式计算“归一化误差”，第二个公式把误差沿通道相关方向传播到剩余列。

$W_{:,i}$ 表示从第 i 列到末尾的所有列， $U_{i,:}$ 是 U 的第 i 行从第 i 列到末尾的元素。更新后，剩余列吸收了第 i 列的量化误差，使后续量化时能“预补偿”这个误差。

直觉 误差补偿像“债务分摊”：第 i 列量化产生了误差“债务”，Hessian 逆矩阵告诉你每个通道应该承担多少。把债务分摊到剩余列后，后续列在量化时会自动“还债”，整体输出误差更小。

6 Cholesky 分解

6.1 为什么需要 Cholesky

误差补偿需要 H^{-1} ，但直接求逆数值不稳定且计算量大。Cholesky 分解把对称正定矩阵 H 分解为 $H = LL^T$ (L 为下三角)，则 $H^{-1} = (L^{-1})^T L^{-1}$ 。令 $U = L^T$ (上三角)，则 $H^{-1} = U^T U$ 。

6.2 优势

Cholesky 分解的数值稳定性优于直接求逆，且上三角矩阵 U 的求逆可以逐行递推，计算量为 $O(\frac{d^3}{3})$ ，比直接求逆的 $O(d^3)$ 快三倍。更重要的是， U 的对角元素 $U_{i,i}$ 可以判断通道的活跃程度：如果 $U_{i,i}$ 接近零，说明该通道在校准数据中几乎不活跃 (“dead column”)。

7 阻尼与数值稳定性

7.1 问题

如果某些通道在校准数据中完全不活跃

($X_{:,j} = 0$)，则 $H_{(j,j)} = 0$ ，Hessian 奇异，Cholesky 分解会失败。即使不完全为零，接近零的对角元素也会导致 $U_{(i,i)}$ 极小，补偿时 $s_{e,i} = (w_i - \text{解决方案})$ 向 Hessian 对角线加入阻尼项：
\$ H <- H + \lambda I \$
其中 $\lambda = \alpha \cdot (1/N \sum_i H_{(i,i)})$ ， α 是阻尼比例（本实验中 `damp_percent = 0.01`）。阻尼项把所有对角元素抬高一个水平，避免 #aside[阻尼比例 $\alpha = 0.01$ 是经验值。太大会模糊通道间的重要性差异，太小无法防止数值爆炸。0.01 在多个模型上表现良好。]
= Dead Column 处理
#v(0.5em)
Dead column 指对角元素为零或极小的列，对应校准数据中完全不活跃的通道。GPTQ 对 dead column 的处理策略是：
#v(0.5em)
+ 检测 $H_{(j,j)} \approx 0$ 的列，标记为 dead column。
+ Dead column 的量化误差为零（因为 $X_{($
0 意味着该列不影响输出），所以可以跳过补偿。

1. 优先处理 dead column，避免它们的零对角元素干扰后续 Cholesky 分解。

框架返回的 metadata 中包含 dead_columns 字段，记录被检测到的 dead column 数量。

8 分块计算

8.1 动机

当权重矩阵的列数 d_{in} 很大（Gemma4 中 $d_{\text{in}} = 3840$ 或 15360 ）时，逐列量化的循环开销和中间矩阵的显存占用都很大。分块计算把列分成若干 block，block 内逐列量化，block 间批量更新。

8.2 策略

以 block_size = 128 列为一个 block：

- **Block 内**：逐列量化，每列量化后立即更新 block 内剩余列。这部分使用 Cholesky 分解的对应子块。
- **Block 外**：一个 block 全部量化完成后，一次性更新所有后续 block。这部分是矩阵乘法，可以利用 BLAS 加速。

分块计算把 $O(d_{\text{in}}^2)$ 的中间矩阵显存降为 $O(d_{\text{in}} \times \text{block_size})$ ，在保持精度的同时大幅降低显存开销。

9 逐层量化流程

GPTQ 对每一层线性层独立执行以下步骤：

1. **收集激活**：用校准数据前向传播，通过 hook 捕获该层的输入激活 X ，形状为 (N, d_{in}) 。
2. **构造 Hessian**：计算 $H = \frac{2}{N} X^T X$ ，加入阻尼 $H \leftarrow H + \lambda I$ 。
3. **Cholesky 分解**：计算 $H^{-1} = U^T U$ ，得到上三角矩阵 U 。
4. **检测 dead column**：标记 $H_{j,j} \approx 0$ 的列。
5. **分块量化**：按 `block_size` 分块，block 内逐列量化并补偿，block 外批量更新。
6. **打包存储**：将量化后的 INT4 权重打包为 `uint8_little_nibble` 格式。
7. **误差传播**：该层的量化误差会通过输出影响下一层的输入激活，但 GPTQ 假设逐层独立，不显式传播。

严格来说，量化第 i 层会改变第 $i+1$ 层的输入激活，但重新收集激活需要再次前向传播，开销大。标准 GPTQ 使用原始激活近似，精度损失可接受。

10 代码接口

框架中 `quantize_weight_gptq` 函数的接口如下：

```
def quantize_weight_gptq(
    weight: torch.Tensor,          # (out_features, in_features)
    activations: torch.Tensor,     # (calibration_tokens, in_features)
    group_size: int,              # 量化粒度
    *,
    block_size: int = 128,        # 分块计算列数
    damp_percent: float = 0.01,   # Hessian 阻尼比例
    symmetric: bool = True,       # 对称量化
    scale_dtype: torch.dtype = torch.float16,
) -> tuple[QuantizedWeight, dict[str, float | int]]:
    ...
```

`weight` 形状为 $(\text{out_features}, \text{in_features})$ ，`activations` 是校准数据前向传播时收集的输入激活。返回值是一个元组：量化后的 `QuantizedWeight` 和一个 `metadata` 字典。`metadata` 包含以下字段：

字段	含义
<code>activation_tokens</code>	校准激活的 token 数
<code>block_size</code>	分块大小
<code>damp_percent</code>	阻尼比例
<code>dead_columns</code>	检测到的 dead column 数
<code>predicted_loss</code>	预测的量化损失

11 实战：手动算 GPTQ

我们用一个 2×2 的权重矩阵和 2×2 的校准激活，手动完成 GPTQ 的全过程，并与 RTN 对比。

例 设定：权重 $W = \begin{pmatrix} 0.3 & 0.5 \\ 0.8 & -0.2 \end{pmatrix}$ ，激活 $X = \begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix}$ ， $N = 2$ ，对称量化，group_size = 2（整个矩阵为一组）。

第 1 步：构造 Hessian

$$X^T X = \begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix}^T \begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix} = \begin{pmatrix} 5 & 5 \\ 5 & 10 \end{pmatrix} \quad (11.1)$$

$$H = \frac{2}{N} X^T X = \begin{pmatrix} 5 & 5 \\ 5 & 10 \end{pmatrix} \quad (11.2)$$

第 2 步：阻尼

$$\lambda = 0.01 \times \frac{5 + 10}{2} = 0.01 \times 7.5 = 0.075 \quad (11.3)$$

$$H + \lambda I = \begin{pmatrix} 5.075 & 5 \\ 5 & 10.075 \end{pmatrix} \quad (11.4)$$

第 3 步：Cholesky 分解 $H + \lambda I = U^T U$

$$U = \begin{pmatrix} 2.253 & 2.219 \\ 0 & 2.270 \end{pmatrix} \quad (11.5)$$

其中 $u_{1,1} = \sqrt{5.075} = 2.253$ ， $u_{1,2} = \frac{5}{2.253} = 2.219$ ， $u_{2,2} = \sqrt{10.075 - 2.219^2} = 2.270$ 。

第 4 步：计算缩放因子

$$s = \frac{\max(|W|)}{7} = \frac{0.8}{7} = 0.1143 \quad (11.6)$$

第 5 步：量化第 1 列

$$w_1 = (0.3, 0.8)^T$$

$$q_1 = \text{round}\left(\frac{w_1}{s}\right) = \text{round}((2.625, 7.0)^T) = (3, 7)^T \quad (11.7)$$

$$\text{dequant}_1 = s \cdot q_1 = (0.3429, 0.8)^T$$

$$\text{error}_1 = w_1 - \text{dequant}_1 = (-0.0429, 0)^T$$

$$e_1 = \frac{\text{error}_1}{u_{1,1}} = (-0.0190, 0)^T \quad (11.8)$$

第 6 步：补偿第 2 列

$$w_2 \leftarrow w_2 - e_1 \cdot u_{1,2} = (0.5, -0.2)^T - (-0.0190, 0)^T \times 2.219 = (0.5422, -0.2)^T$$

第 7 步：量化第 2 列

$$q_2 = \text{round}\left(\frac{w_2}{s}\right) = \text{round}((4.74, -1.75)^T) = (5, -2)^T \quad (11.9)$$

$$\text{dequant}_2 = s \cdot q_2 = (0.5714, -0.2286)^T$$

$$\text{GPTQ 结果：} q = \begin{pmatrix} 3 & 5 \\ 7 & -2 \end{pmatrix}, \text{ dequant} = \begin{pmatrix} 0.3429 & 0.5714 \\ 0.8 & -0.2286 \end{pmatrix}$$

RTN 结果（无补偿，直接量化第 2 列）：

$$q_{\text{RTN},2} = \text{round}\left(\frac{(0.5, -0.2)^T}{0.1143}\right) = (4, -2)^T \quad (11.10)$$

$$\text{dequant}_{\text{RTN},2} = (0.4571, -0.2286)^T$$

$$q_{\text{RTN}} = \begin{pmatrix} 3 & 4 \\ 7 & -2 \end{pmatrix}$$

输出误差对比 ($Y = XW$):

$$\text{原始输出: } Y = \begin{pmatrix} 1.4 & 0.8 \\ 2.7 & -0.1 \end{pmatrix}$$

$$\text{RTN 输出: } \begin{pmatrix} 1.4858 & 0.6857 \\ 2.7429 & -0.2286 \end{pmatrix}, \text{ 误差平方和} = 0.0388$$

$$\text{GPTQ 输出: } \begin{pmatrix} 1.4858 & 0.9143 \\ 2.7429 & -0.1143 \end{pmatrix}, \text{ 误差平方和} = 0.0225$$

GPTQ 把输出误差平方和从 0.0388 降到 0.0225，减少 42%。关键在第 2 列：RTN 量化为 4 (0.4571)，误差 0.1143；GPTQ 补偿后量化为 5 (0.5714)，误差 -0.1143。虽然 q_2 从 4 变成了 5，但结合第 1 列的误差，整体输出更接近原始值。

12 本章你将学会

1. 解释 RTN 的根本缺陷：独立量化每个权重，不考虑输入分布和通道间相关性。
2. 写出层重构目标 $\min_Q \|XW^T - XQ^T\|_F^2$ ，推导 Hessian 矩阵 $H = \frac{2}{N}X^T X$ 。
3. 描述误差补偿公式 $e_i = \frac{w_i - q_i}{U_{i,i}}$ 和 $W_{:,i} \leftarrow W_{:,i} - e_i U_{i,:}$ ，解释每个符号。
4. 说明 Cholesky 分解的作用和优势，以及阻尼项 $H \leftarrow H + \lambda I$ 的必要性。
5. 解释 dead column 的成因和处理方式。
6. 手动完成 2×2 矩阵的 GPTQ 全过程，计算 Hessian、Cholesky、误差补偿，并与 RTN 对比输出误差。

13 小结

GPTQ 用二阶信息补偿量化误差：逐列量化权重后，通过 Hessian 逆矩阵的 Cholesky 分解把误差传播到剩余列，使整体输出误差最小化。阻尼项保证数值稳定性，分块计算降低显存开销，dead column 检测跳过不活跃通道。在 2×2 的例子中，GPTQ 把输出误差平方和减少了 42%。下一章我们将分析 LLM 推理的完整显存构成，理解为什么 INT4 量化是 10 GiB 显存约束下的必选项。

讲义基于 HPC Lab5 实验指导编写